

Modelos de regresión en Machine Learning

Edison Achalma

Escuela Profesional de Economía, Universidad Nacional de San Cristóbal de Huamanga

Resumen

Este abstract será actualizado una vez que se complete el contenido final del artículo.

Palabras Claves: keyword1, keyword2

Tabla de contenidos

Introduction	3
1 La regresión	3
1.1 Concepto	3
1.2 Tipos de problemas de regresión	3
1.3 Generar datos de ejemplo	4
1.4 Dividir datos	5
2 Mínimos cuadrados ordinarios (OLS)	5
2.1 Fundamento matemático	5
2.1.1 Especificación del modelo	5
2.1.2 Función objetivo	6
2.1.3 Derivación de la solución	6
2.1.4 Propiedades del estimador OLS	7
2.1.5 Matriz de varianzas-covarianzas	7
2.1.6 Supuestos del modelo clásico de regresión lineal	7
2.1.7 Ventajas del método OLS	8
2.1.8 Limitaciones del método OLS	9
2.1.9 Aplicaciones en economía	9
2.1.10 Consideraciones computacionales	9
2.2 Implementación	10
2.3 Visualización de predicciones	10
2.4 Análisis de residuos	11

Edison Achalma  <https://orcid.org/0000-0001-6996-3364>

El autor no tiene conflictos de interés que revelar. Los roles de autor se clasificaron utilizando la taxonomía de roles de colaborador (CRediT; <https://credit.niso.org/>) de la siguiente manera: Edison Achalma: conceptualización, metodología, análisis formal, investigación, recursos, curación de datos, redacción, visualización, supervisión, administración del proyecto

La correspondencia relativa a este artículo debe dirigirse a Edison Achalma, Escuela Profesional de Economía, Universidad Nacional de San Cristóbal de Huamanga, Portal Independencia N° 57, Ayacucho, AYA 5001, Perú, Email: elmer.achalma.09@unsch.edu.pe

3	Regresión Ridge	12
3.1	Fundamento matemático	12
3.2	Implementación	13
3.3	Visualización del efecto de regularización	13
4	Regresión Lasso	14
4.1	Fundamento matemático	14
4.2	Implementación	15
4.3	Comparación Ridge vs Lasso	16
5	Elastic Net	17
5.1	Fundamento	17
5.2	Implementación	18
6	Regresión polinomial	18
6.1	Fundamento	18
6.2	Implementación	19
7	K-Nearest Neighbors Regressor	21
7.1	Fundamento	21
7.2	Implementación	21
8	Métricas de evaluación	23
8.1	Explicación de métricas	23
8.2	Implementación de métricas	24
9	Comparación de modelos	25
9.1	Tabla y visualización comparativa	25
10	Aplicaciones económicas	26
10.1	Aplicación 1: Predicción de precios de vivienda	26
10.2	Aplicación 2: Predicción de ventas	29
11	Ejercicios prácticos	32
11.1	Ejercicio: Validación cruzada	32
12	Conclusión	34
13	Próximos pasos	34
14	Recursos adicionales	34
15	Publicaciones Similares	34

Modelos de regresión en Machine Learning

En esta décima guía exploraremos los principales algoritmos de machine learning para problemas de regresión. Estos métodos son fundamentales para resolver problemas económicos como predicción de precios, estimación de demanda, forecasting de series de tiempo, valuación de activos, entre otros.

1 La regresión

1.1 Concepto

La regresión es un método para predecir valores numéricos continuos (a diferencia de clasificación que predice categorías discretas).

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Scikit-learn
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet
from sklearn.neighbors import KNeighborsRegressor
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

# Configuración
plt.style.use('seaborn-v0_8-whitegrid')
plt.rcParams['figure.figsize'] = (12, 6)
np.random.seed(42)

print("Librerías importadas exitosamente")
```

1.2 Tipos de problemas de regresión

1. Predicción de precios

- Precios de viviendas
- Precios de acciones
- Precios de commodities
- Tipos de cambio

2. Estimación de demanda*

- Demanda de productos
- Demanda laboral
- Demanda de servicios públicos

3. Forecasting económico (pronósticos)

- PIB futuro
- Ventas futuras
- Inflación
- Tasas de interés

4. Valuación

- Valuación de empresas
- Valuación de activos
- Riesgo de crédito (score continuo)

5. Análisis de impacto y causalidad

- Efecto de políticas públicas
- Retorno de inversión en publicidad
- Elasticidades (precio, ingreso)

1.3 Generar datos de ejemplo

```
def generar_datos_regresion(n=200, ruido=1.0, seed=42):  
    """  
    Genera datos sintéticos para regresión  
  
    Modelo:  $Y = 3 + 2 \cdot X_1 - 1.5 \cdot X_2 +$   
    """  
    np.random.seed(seed)  
  
    # Variables explicativas  
    X = np.random.randn(n, 2)  
  
    # Coeficientes verdaderos  
    beta_true = np.array([2.0, -1.5])  
    intercepto_true = 3.0  
  
    # Variable dependiente con ruido  
    Y = intercepto_true + np.dot(X, beta_true) + np.random.randn(n) * ruido  
  
    return X, Y, beta_true, intercepto_true  
  
# Generar datos  
X, Y, beta_true, intercepto_true = generar_datos_regresion(n=200, ruido=1.0)  
  
print("DATOS GENERADOS")  
print("=" * 70)  
print(f"Número de observaciones: {len(Y)}")  
print(f"Número de características: {X.shape[1]}")  
print(f"\nCoeficientes verdaderos:")  
print(f"    Intercepto: {intercepto_true}")
```

```

print(f"      : {beta_true[0]}")
print(f"      : {beta_true[1]}")
print(f"\nModelo verdadero: Y = {intercepto_true} + {beta_true[0]}*X + {beta_true[1]}")

# Visualizar datos
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].scatter(X[:, 0], Y, alpha=0.6, s=50)
axes[0].set_xlabel('X ')
axes[0].set_ylabel('Y')
axes[0].set_title('Relación Y vs X ')
axes[0].grid(True, alpha=0.3)

axes[1].scatter(X[:, 1], Y, alpha=0.6, s=50, color='orange')
axes[1].set_xlabel('X ')
axes[1].set_ylabel('Y')
axes[1].set_title('Relación Y vs X ')
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('datos_regresion.png', dpi=300, bbox_inches='tight')
print("\nGráfico guardado como 'datos_regresion.png'")

```

1.4 Dividir datos

```

# Dividir en entrenamiento y prueba
X_train, X_test, y_train, y_test = train_test_split(
    X, Y, test_size=0.3, random_state=42
)

print(f"\nDivisión de datos:")
print(f"  Entrenamiento: {len(y_train)} observaciones")
print(f"  Prueba: {len(y_test)} observaciones")

```

2 Mínimos cuadrados ordinarios (OLS)

2.1 Fundamento matemático

El método de Mínimos Cuadrados Ordinarios (Ordinary Least Squares, OLS) es el método más fundamental y ampliamente utilizado en econometría y análisis de regresión. Fue desarrollado por Carl Friedrich Gauss a principios del siglo XIX y constituye la base para muchos métodos estadísticos modernos.

2.1.1 Especificación del modelo

El modelo de regresión lineal múltiple se especifica como:

$$Y_i = \beta_0 + \beta_1 X_{1i} + \beta_2 X_{2i} + \cdots + \beta_p X_{pi} + \varepsilon_i$$

donde:

- Y_i es el valor observado de la variable dependiente para la observación i
- X_{ji} es el valor de la j -ésima variable independiente para la observación i
- β_0 es el intercepto (término constante)
- β_j son los coeficientes de regresión para $j = 1, 2, \dots, p$
- ε_i es el término de error aleatorio
- $i = 1, 2, \dots, n$ indexa las observaciones

En notación matricial, el modelo se expresa de forma más compacta como:

$$\mathbf{Y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}$$

donde:

- $\mathbf{Y}$ es un vector $n \times 1$ de observaciones de la variable dependiente
- $\mathbf{X}$ es una matriz $n \times (p + 1)$ de variables independientes (incluye columna de unos para el intercepto)
- $\boldsymbol{\beta}$ es un vector $(p + 1) \times 1$ de coeficientes
- $\boldsymbol{\varepsilon}$ es un vector $n \times 1$ de errores

2.1.2 Función objetivo

El método OLS busca estimar los coeficientes $\boldsymbol{\beta}$ que minimizan la Suma de Residuos al Cuadrado (Residual Sum of Squares, RSS):

$$\text{RSS}(\boldsymbol{\beta}) = \sum_{i=1}^n \varepsilon_i^2 = \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$$

donde $\hat{Y}_i = \beta_0 + \beta_1 X_{1i} + \dots + \beta_p X_{pi}$ es el valor predicho.

Expandiendo la expresión:

$$\text{RSS}(\boldsymbol{\beta}) = \sum_{i=1}^n \left(Y_i - \beta_0 - \sum_{j=1}^p \beta_j X_{ji} \right)^2$$

En notación matricial:

$$\text{RSS}(\boldsymbol{\beta}) = (\mathbf{Y} - \mathbf{X}\boldsymbol{\beta})^\top (\mathbf{Y} - \mathbf{X}\boldsymbol{\beta})$$

2.1.3 Derivación de la solución

Para encontrar el estimador que minimiza RSS, tomamos la derivada con respecto a $\boldsymbol{\beta}$ e igualamos a cero:

$$\frac{\partial \text{RSS}}{\partial \boldsymbol{\beta}} = -2\mathbf{X}^\top (\mathbf{Y} - \mathbf{X}\boldsymbol{\beta}) = 0$$

Resolviendo para $\boldsymbol{\beta}$:

$$\mathbf{X}^\top \mathbf{Y} = \mathbf{X}^\top \mathbf{X} \boldsymbol{\beta}$$

Estas son las **ecuaciones normales** del problema de mínimos cuadrados. Asumiendo que $\mathbf{X}^\top \mathbf{X}$ es invertible (es decir, las columnas de $\mathbf{X}$ son linealmente independientes), la solución es:

$$\hat{\beta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{Y}$$

Esta es la **solución de mínimos cuadrados ordinarios**.

2.1.4 Propiedades del estimador OLS

Bajo los supuestos estándar (ver sección siguiente), el estimador OLS posee las siguientes propiedades deseables:

1. **Insesgadez:** El estimador es insesgado, es decir:

$$E[\hat{\beta}] = \beta$$

2. **Varianza mínima:** Según el Teorema de Gauss-Markov, OLS es el estimador lineal insesgado de varianza mínima (BLUE: Best Linear Unbiased Estimator).
3. **Consistencia:** El estimador converge en probabilidad al verdadero valor cuando $n \rightarrow \infty$:

$$\text{plim}_{n \rightarrow \infty} \hat{\beta} = \beta$$

4. **Normalidad asintótica:** Bajo condiciones regulares:

$$\sqrt{n}(\hat{\beta} - \beta) \xrightarrow{d} N(0, \sigma^2(\mathbf{X}^\top \mathbf{X})^{-1})$$

2.1.5 Matriz de varianzas-covarianzas

La matriz de varianzas-covarianzas del estimador OLS está dada por:

$$\text{Var}(\hat{\beta}) = \sigma^2(\mathbf{X}^\top \mathbf{X})^{-1}$$

donde $\sigma^2 = \text{Var}(\varepsilon_i)$ es la varianza del error. En la práctica, σ^2 se estima mediante:

$$\hat{\sigma}^2 = \frac{\text{RSS}}{n - p - 1} = \frac{\sum_{i=1}^n \hat{\varepsilon}_i^2}{n - p - 1}$$

Los errores estándar de los coeficientes son las raíces cuadradas de los elementos diagonales de $\hat{\sigma}^2(\mathbf{X}^\top \mathbf{X})^{-1}$.

2.1.6 Supuestos del modelo clásico de regresión lineal

Para que el estimador OLS tenga las propiedades óptimas mencionadas, se requieren los siguientes supuestos:

Supuesto 1: Linealidad

El modelo es lineal en los parámetros. La verdadera relación entre Y y las X puede expresarse como:

$$E[Y_i | X_{1i}, \dots, X_{pi}] = \beta_0 + \beta_1 X_{1i} + \dots + \beta_p X_{pi}$$

Supuesto 2: Exogeneidad estricta

Los errores tienen media condicional cero:

$$E[\varepsilon_i | \mathbf{X}] = 0 \quad \text{para todo } i$$

Esto implica que las variables independientes no están correlacionadas con el término de error.

Supuesto 3: No colinealidad perfecta

Las columnas de $\mathbf{X}$ son linealmente independientes, es decir, la matriz $\mathbf{X}^\top \mathbf{X}$ es de rango completo y por tanto invertible. No existe $\mathbf{a} \neq \mathbf{0}$ tal que $\mathbf{Xa} = \mathbf{0}$.

Supuesto 4: Homocedasticidad

Los errores tienen varianza constante:

$$\text{Var}(\varepsilon_i | \mathbf{X}) = \sigma^2 \quad \text{para todo } i$$

Supuesto 5: No autocorrelación

Los errores no están correlacionados entre sí:

$$\text{Cov}(\varepsilon_i, \varepsilon_j | \mathbf{X}) = 0 \quad \text{para } i \neq j$$

Los supuestos 4 y 5 pueden combinarse como:

$$E[\boldsymbol{\varepsilon} \boldsymbol{\varepsilon}^\top | \mathbf{X}] = \sigma^2 \mathbf{I}_n$$

Supuesto 6: Normalidad (opcional)

Para inferencia en muestras finitas, se asume:

$$\varepsilon_i | \mathbf{X} \sim N(0, \sigma^2)$$

Este supuesto no es necesario para propiedades asintóticas pero permite realizar pruebas de hipótesis exactas en muestras pequeñas.

2.1.7 Ventajas del método OLS

- **Simplicidad conceptual e implementación:** El método tiene una interpretación geométrica clara (proyección ortogonal) y es computacionalmente eficiente.
- **Interpretabilidad:** Los coeficientes tienen interpretación directa como efectos marginales. β_j representa el cambio en Y por un cambio unitario en X_j , manteniendo constantes las demás variables.
- **Solución cerrada:** Existe una fórmula analítica explícita para el estimador, lo que facilita el análisis teórico.
- **Propiedades estadísticas óptimas:** Bajo los supuestos del modelo clásico, OLS es BLUE.
- **Base para extensiones:** OLS es el fundamento de métodos más sofisticados como mínimos cuadrados generalizados (GLS), variables instrumentales (IV), y modelos de panel.
- **Inferencia estadística:** Permite construir intervalos de confianza y realizar pruebas de hipótesis sobre los coeficientes.

2.1.8 Limitaciones del método OLS

- **Sensibilidad a violaciones de supuestos:**
 - La **heterocedasticidad** (varianza no constante) invalida los errores estándar estándar
 - La **autocorrelación** (común en series de tiempo) sesga los errores estándar
 - La **endogeneidad** (correlación entre X y ε) produce estimadores sesgados e inconsistentes
- **Sensibilidad a outliers:** Valores extremos pueden ejercer influencia desproporcionada en las estimaciones debido a la minimización de errores al cuadrado.
- **Multicolinealidad:** Cuando las variables independientes están altamente correlacionadas:
 - Los errores estándar de los coeficientes se inflan
 - Los coeficientes individuales se vuelven inestables
 - La matriz $\mathbf{X}^T \mathbf{X}$ se aproxima a singularidad
- **Restricción a modelos lineales:** OLS solo puede capturar relaciones lineales en los parámetros (aunque puede incluir transformaciones no lineales de las variables).
- **No realiza selección de variables:** Incluye todas las variables especificadas sin discriminar su relevancia.
- **Overfitting con muchas variables:** En modelos con p grande relativo a n , puede ajustar demasiado al ruido de los datos de entrenamiento, deteriorando la capacidad predictiva fuera de muestra.

2.1.9 Aplicaciones en economía

El método OLS es ubicuo en economía y se aplica a:

- **Estimación de funciones de demanda:** Relación entre cantidad demandada, precio, ingreso y otros factores
- **Funciones de producción:** Relación entre insumos (capital, trabajo) y producto
- **Ecuaciones de salarios:** Retornos a la educación y experiencia (ecuación de Mincer)
- **Modelos de consumo:** Propensión marginal a consumir
- **Análisis de series macroeconómicas:** PIB, inflación, desempleo
- **Evaluación de políticas:** Efectos de programas sociales o intervenciones

2.1.10 Consideraciones computacionales

La inversión de la matriz $\mathbf{X}^T \mathbf{X}$ es computacionalmente costosa cuando p es grande. Métodos numéricos estables incluyen:

- **Descomposición QR:** $\mathbf{X} = \mathbf{QR}$, luego $\hat{\beta} = \mathbf{R}^{-1} \mathbf{Q}^T \mathbf{Y}$
- **Descomposición SVD:** Descomposición en valores singulares de $\mathbf{X}$
- **Eliminación gaussiana:** Para resolver directamente las ecuaciones normales

En Python, `sklearn.linear_model.LinearRegression` utiliza descomposición SVD por defecto, lo que proporciona estabilidad numérica superior.

2.2 Implementación

```
# Entrenar modelo OLS
modelo_ols = LinearRegression()
modelo_ols.fit(X_train, y_train)

# Coeficientes estimados
print("\nMODELO OLS ESTIMADO")
print("=" * 70)
print(f"Intercepto estimado: {modelo_ols.intercept_:.4f}")
print(f"  (Verdadero: {intercepto_true})")
print(f"\nCoeficientes estimados:")
print(f"    : {modelo_ols.coef_[0]:.4f} (Verdadero: {beta_true[0]})")
print(f"    : {modelo_ols.coef_[1]:.4f} (Verdadero: {beta_true[1]})")

# Predecir
y_pred_ols_train = modelo_ols.predict(X_train)
y_pred_ols_test = modelo_ols.predict(X_test)

# Evaluar
r2_train = r2_score(y_train, y_pred_ols_train)
r2_test = r2_score(y_test, y_pred_ols_test)
mse_train = mean_squared_error(y_train, y_pred_ols_train)
mse_test = mean_squared_error(y_test, y_pred_ols_test)

print(f"\n\nPERFORMANCE DEL MODELO")
print("=" * 70)
print(f"R2 Train: {r2_train:.4f}")
print(f"R2 Test: {r2_test:.4f}")
print(f"MSE Train: {mse_train:.4f}")
print(f"MSE Test: {mse_test:.4f}")
```

2.3 Visualización de predicciones

```
# Gráfico de valores reales vs predichos
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Training set
axes[0].scatter(y_train, y_pred_ols_train, alpha=0.6, s=50)
axes[0].plot([y_train.min(), y_train.max()],
             [y_train.min(), y_train.max()],
             'r--', linewidth=2, label='Predicción perfecta')
axes[0].set_xlabel('Y real')
axes[0].set_ylabel('Y predicho')
axes[0].set_title(f'OLS - Entrenamiento (R2= {r2_train:.3f})')
axes[0].legend()
axes[0].grid(True, alpha=0.3)
```

```
# Test set
axes[1].scatter(y_test, y_pred_ols_test, alpha=0.6, s=50, color='orange')
axes[1].plot([y_test.min(), y_test.max()],
             [y_test.min(), y_test.max()],
             'r--', linewidth=2, label='Predicción perfecta')
axes[1].set_xlabel('Y real')
axes[1].set_ylabel('Y predicho')
axes[1].set_title(f'OLS - Prueba ( $R^2$ ={r2_test:.3f})')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('ols_predictions.png', dpi=300, bbox_inches='tight')
print("\nGráfico guardado como 'ols_predictions.png'")
```

2.4 Análisis de residuos

```
# Calcular residuos
residuos_train = y_train - y_pred_ols_train
residuos_test = y_test - y_pred_ols_test

# Gráficos de diagnóstico
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# 1. Residuos vs valores predichos
axes[0, 0].scatter(y_pred_ols_train, residuos_train, alpha=0.6)
axes[0, 0].axhline(y=0, color='r', linestyle='--', linewidth=2)
axes[0, 0].set_xlabel('Valores predichos')
axes[0, 0].set_ylabel('Residuos')
axes[0, 0].set_title('Residuos vs Predicciones')
axes[0, 0].grid(True, alpha=0.3)

# 2. Histograma de residuos
axes[0, 1].hist(residuos_train, bins=30, edgecolor='black', alpha=0.7)
axes[0, 1].axvline(x=0, color='r', linestyle='--', linewidth=2)
axes[0, 1].set_xlabel('Residuos')
axes[0, 1].set_ylabel('Frecuencia')
axes[0, 1].set_title('Distribución de Residuos')
axes[0, 1].grid(True, alpha=0.3)

# 3. Q-Q plot
from scipy import stats
stats.probplot(residuos_train, dist="norm", plot=axes[1, 0])
axes[1, 0].set_title('Q-Q Plot')
axes[1, 0].grid(True, alpha=0.3)
```

```
# 4. Residuos vs orden
axes[1, 1].scatter(range(len(residuos_train)), residuos_train, alpha=0.6)
axes[1, 1].axhline(y=0, color='r', linestyle='--', linewidth=2)
axes[1, 1].set_xlabel('Índice de observación')
axes[1, 1].set_ylabel('Residuos')
axes[1, 1].set_title('Residuos vs Orden')
axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('ols_diagnostics.png', dpi=300, bbox_inches='tight')
print("Gráficos de diagnóstico guardados como 'ols_diagnostics.png'")
```

3 Regresión Ridge

3.1 Fundamento matemático

```
print("""
REGRESIÓN RIDGE (L2 REGULARIZATION)
```

Objetivo:

Minimizar RSS + penalización por magnitud de coeficientes

$$\min \{ \text{RSS}() + \lambda \sum \beta_j^2 \}$$

$$= \min \{ \sum (y_i - \hat{y}_i)^2 + \lambda \sum \beta_j^2 \}$$

Solución:

$$\hat{\beta} = (X'X + \lambda I)^{-1} X'Y$$

Donde:

- λ : Parámetro de regularización ($\lambda \geq 0$)
 - * $\lambda = 0$: Equivalente a OLS
 - * $\lambda \rightarrow \infty$: Coeficientes $\rightarrow 0$
- $\|\beta\|_2^2$: Norma L2 (suma de cuadrados de coeficientes)

Efecto de Ridge:

- Reduce magnitud de coeficientes (shrinkage)
- Mantiene todas las variables (no hace selección)
- Ayuda con multicolinealidad
- Reduce varianza del modelo
- Previene overfitting

Cuándo usar Ridge:

- Cuando hay muchas variables correlacionadas
- Cuando todas las variables son potencialmente relevantes
- Cuando se prefiere estabilidad sobre interpretabilidad

Aplicaciones económicas:

- Modelos con muchas variables macroeconómicas correlacionadas
 - Predicción de series de tiempo con múltiples rezagos
 - Modelos de pricing con características correlacionadas
- """)

3.2 Implementación

```
# Probar diferentes valores de alpha
alphas = [0.001, 0.01, 0.1, 1, 10, 100, 1000]

print("\nREGRESIÓN RIDGE - EFECTO DE ")
print("=" * 70)
print(f"{' ':<10} {' ':<12} {' ':<12} {'R² Train':<12} {'R² Test':<12}")
print("-" * 70)

resultados_ridge = []

for alpha in alphas:
    # Entrenar modelo
    modelo_ridge = Ridge(alpha=alpha)
    modelo_ridge.fit(X_train, y_train)

    # Evaluar
    r2_train = modelo_ridge.score(X_train, y_train)
    r2_test = modelo_ridge.score(X_test, y_test)

    resultados_ridge.append({
        'alpha': alpha,
        'beta1': modelo_ridge.coef_[0],
        'beta2': modelo_ridge.coef_[1],
        'r2_train': r2_train,
        'r2_test': r2_test
    })

    print(f"{alpha:<10.3f} {modelo_ridge.coef_[0]:<12.4f} "
          f"{modelo_ridge.coef_[1]:<12.4f} {r2_train:<12.4f} {r2_test:<12.4f}")

df_ridge = pd.DataFrame(resultados_ridge)
```

3.3 Visualización del efecto de regularización

```
# Gráfico de coeficientes vs alpha
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Coeficientes
```

```

axes[0].semilogx(df_ridge['alpha'], df_ridge['beta1'],
                 'o-', linewidth=2, markersize=8, label=' ')
axes[0].semilogx(df_ridge['alpha'], df_ridge['beta2'],
                 's-', linewidth=2, markersize=8, label=' ')
axes[0].axhline(y=beta_true[0], color='blue', linestyle='--',
                alpha=0.5, label=' verdadero')
axes[0].axhline(y=beta_true[1], color='orange', linestyle='--',
                alpha=0.5, label=' verdadero')
axes[0].set_xlabel(' (escala logarítmica)')
axes[0].set_ylabel('Valor del coeficiente')
axes[0].set_title('Coeficientes vs Parámetro de Regularización')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# R²
axes[1].semilogx(df_ridge['alpha'], df_ridge['r2_train'],
                 'o-', linewidth=2, markersize=8, label='Train')
axes[1].semilogx(df_ridge['alpha'], df_ridge['r2_test'],
                 's-', linewidth=2, markersize=8, label='Test')
axes[1].set_xlabel(' (escala logarítmica)')
axes[1].set_ylabel('R²')
axes[1].set_title('R² vs Parámetro de Regularización')
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('ridge_regularization.png', dpi=300, bbox_inches='tight')
print("\nGráfico guardado como 'ridge_regularization.png'")

# Mejor alpha
mejor_idx = df_ridge['r2_test'].idxmax()
mejor_alpha = df_ridge.loc[mejor_idx, 'alpha']
print(f"\nMejor : {mejor_alpha}")
print(f"R² Test: {df_ridge.loc[mejor_idx, 'r2_test']:.4f}")

```

4 Regresión Lasso

4.1 Fundamento matemático

```

print("""
REGRESIÓN LASSO (L1 REGULARIZATION)

Objetivo:
    Minimizar RSS + penalización por valor absoluto de coeficientes

    min { RSS() + ·Σ| | }

```

$$= \min \{ \sum (y - \hat{y})^2 + \lambda \sum | \beta_j | \}$$

Donde:

- λ : Parámetro de regularización ($\lambda \geq 0$)
- $\sum | \beta_j |$: Norma L1 (suma de valores absolutos)

Diferencia clave con Ridge:

- Lasso puede llevar coeficientes exactamente a CERO
- Ridge solo los reduce pero no los elimina

Efecto de Lasso:

- Selección automática de variables (feature selection)
- Produce modelos dispersos (sparse models)
- Más interpretable que Ridge
- Útil cuando hay muchas variables irrelevantes

Cuándo usar Lasso:

- Cuando se cree que solo pocas variables son relevantes
- Cuando se necesita interpretabilidad
- Cuando se quiere selección automática de variables

Aplicaciones económicas:

- Identificar factores clave que afectan un outcome
 - Modelos parsimoniosos para reportes
 - Análisis exploratorio de muchas variables
 - Predicción con datasets con muchas columnas
- """)

4.2 Implementación

```
# Probar diferentes valores de alpha
alphas_lasso = [0.001, 0.01, 0.1, 0.5, 1, 2, 5]

print("\nREGRESIÓN LASSO - EFECTO DE ")
print("=" * 80)
print(f"{' ':<10} {' ':<12} {' ':<12} {'Variables 0':<14} {'R² Train':<12} {'R² Test':<12}")
print("-" * 80)

resultados_lasso = []

for alpha in alphas_lasso:
    # Entrenar modelo
    modelo_lasso = Lasso(alpha=alpha, max_iter=10000)
    modelo_lasso.fit(X_train, y_train)

    # Contar variables no cero
    n_nonzero = np.sum(modelo_lasso.coef_ != 0)
```

```

# Evaluar
r2_train = modelo_lasso.score(X_train, y_train)
r2_test = modelo_lasso.score(X_test, y_test)

resultados_lasso.append({
    'alpha': alpha,
    'beta1': modelo_lasso.coef_[0],
    'beta2': modelo_lasso.coef_[1],
    'n_nonzero': n_nonzero,
    'r2_train': r2_train,
    'r2_test': r2_test
})

print(f"{alpha:<10.3f} {modelo_lasso.coef_[0]:<12.4f} "
      f"{modelo_lasso.coef_[1]:<12.4f} {n_nonzero:<14d} "
      f"{r2_train:<12.4f} {r2_test:<12.4f}")

df_lasso = pd.DataFrame(resultados_lasso)

```

4.3 Comparación Ridge vs Lasso

```

# Visualización comparativa
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# Coeficiente 1
axes[0, 0].semilogx(df_ridge['alpha'], df_ridge['beta1'],
                    'o-', linewidth=2, label='Ridge', markersize=8)
axes[0, 0].semilogx(df_lasso['alpha'], df_lasso['beta1'],
                    's-', linewidth=2, label='Lasso', markersize=8)
axes[0, 0].axhline(y=beta_true[0], color='black', linestyle='--',
                  alpha=0.5, label='Verdadero')
axes[0, 0].set_xlabel(' ')
axes[0, 0].set_ylabel(' ')
axes[0, 0].set_title('Coeficiente vs ')
axes[0, 0].legend()
axes[0, 0].grid(True, alpha=0.3)

# Coeficiente 2
axes[0, 1].semilogx(df_ridge['alpha'], df_ridge['beta2'],
                    'o-', linewidth=2, label='Ridge', markersize=8)
axes[0, 1].semilogx(df_lasso['alpha'], df_lasso['beta2'],
                    's-', linewidth=2, label='Lasso', markersize=8)
axes[0, 1].axhline(y=beta_true[1], color='black', linestyle='--',
                  alpha=0.5, label='Verdadero')
axes[0, 1].set_xlabel(' ')
axes[0, 1].set_ylabel(' ')

```



```

axes[0, 1].set_title('Coeficiente vs ')
axes[0, 1].legend()
axes[0, 1].grid(True, alpha=0.3)

# R2 Train
axes[1, 0].semilogx(df_ridge['alpha'], df_ridge['r2_train'],
                    'o-', linewidth=2, label='Ridge', markersize=8)
axes[1, 0].semilogx(df_lasso['alpha'], df_lasso['r2_train'],
                    's-', linewidth=2, label='Lasso', markersize=8)
axes[1, 0].set_xlabel(' ')
axes[1, 0].set_ylabel('R2')
axes[1, 0].set_title('R2 Entrenamiento vs ')
axes[1, 0].legend()
axes[1, 0].grid(True, alpha=0.3)

# R2 Test
axes[1, 1].semilogx(df_ridge['alpha'], df_ridge['r2_test'],
                    'o-', linewidth=2, label='Ridge', markersize=8)
axes[1, 1].semilogx(df_lasso['alpha'], df_lasso['r2_test'],
                    's-', linewidth=2, label='Lasso', markersize=8)
axes[1, 1].set_xlabel(' ')
axes[1, 1].set_ylabel('R2')
axes[1, 1].set_title('R2 Test vs ')
axes[1, 1].legend()
axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig('ridge_vs_lasso.png', dpi=300, bbox_inches='tight')
print("\nGráfico guardado como 'ridge_vs_lasso.png'")

```

5 Elastic Net

5.1 Fundamento

```

print("""
ELASTIC NET

Combina penalizaciones L1 (Lasso) y L2 (Ridge):

    min { RSS() + λ · Σ|βj| + λ · (1-α)/2 · Σ βj2 }

Donde:
- λ : Fuerza total de regularización
- α : Balance entre L1 y L2 (0 ≤ α ≤ 1)
  * α = 0: Ridge puro
  * α = 1: Lasso puro
  * 0 < α < 1: Combinación

```

Ventajas:

- Combina lo mejor de Ridge y Lasso
- Selección de variables (como Lasso)
- Manejo de variables correlacionadas (como Ridge)
- Más estable que Lasso cuando hay correlaciones altas

Aplicaciones:

- Cuando hay grupos de variables correlacionadas
 - Cuando se necesita selección de variables y estabilidad
- """)

5.2 Implementación

```
# Entrenar Elastic Net
modelo_elastic = ElasticNet(alpha=0.1, l1_ratio=0.5, max_iter=10000)
modelo_elastic.fit(X_train, y_train)

print("\nELASTIC NET")
print("=" * 70)
print(f"Intercepto: {modelo_elastic.intercept_:.4f}")
print(f"Coeficientes:")
print(f"      : {modelo_elastic.coef_[0]:.4f}")
print(f"      : {modelo_elastic.coef_[1]:.4f}")

r2_elastic = modelo_elastic.score(X_test, y_test)
print(f"\nR2 Test: {r2_elastic:.4f}")
```

6 Regresión polinomial

6.1 Fundamento

```
print("""
REGRESIÓN POLINOMIAL
```

Extiende regresión lineal creando características polinomiales:

Grado 1 (lineal): $Y = \quad + X$
 Grado 2: $Y = \quad + X + X^2$
 Grado 3: $Y = \quad + X + X^2 + X^3$

Proceso:

1. Crear características polinomiales (X , X^2 , X^3 , etc.)
2. Aplicar regresión lineal en características transformadas

Ventajas:

- Captura relaciones no lineales

Flexible para ajustar curvas complejas

Desventajas:

- Propenso a overfitting con grados altos
- Inestable en extremos (extrapolación)
- Difícil de interpretar

Aplicaciones económicas:

- Curvas de costo (U invertida)
- Funciones de producción con rendimientos decrecientes
- Curvas de demanda no lineales
- Ciclos económicos

""")

6.2 Implementación

```
# Generar datos con relación no lineal
np.random.seed(42)
X_poly_1d = np.random.uniform(-3, 3, 100)
Y_poly = 0.5 * X_poly_1d**2 - 2 * X_poly_1d + 1 + np.random.randn(100) * 2

# Ordenar para visualización
sort_idx = np.argsort(X_poly_1d)
X_poly_1d = X_poly_1d[sort_idx]
Y_poly = Y_poly[sort_idx]

X_poly_1d = X_poly_1d.reshape(-1, 1)

# Dividir datos
X_train_poly, X_test_poly, y_train_poly, y_test_poly = train_test_split(
    X_poly_1d, Y_poly, test_size=0.3, random_state=42
)

# Probar diferentes grados
grados = [1, 2, 3, 5, 8]
colores = ['blue', 'green', 'red', 'purple', 'orange']

plt.figure(figsize=(14, 8))

print("\nREGRESIÓN POLINOMIAL")
print("-" * 70)
print(f"{'Grado':<8} {'R² Train':<12} {'R² Test':<12} {'MSE Train':<12} {'MSE Test':<12}")
print("-" * 70)

for grado, color in zip(grados, colores):
    # Crear características polinomiales
    poly_features = PolynomialFeatures(degree=grado, include_bias=False)
```

```
X_train_poly_transformed = poly_features.fit_transform(X_train_poly)
X_test_poly_transformed = poly_features.transform(X_test_poly)

# Entrenar modelo
modelo = LinearRegression()
modelo.fit(X_train_poly_transformed, y_train_poly)

# Evaluar
r2_train = modelo.score(X_train_poly_transformed, y_train_poly)
r2_test = modelo.score(X_test_poly_transformed, y_test_poly)

y_pred_train = modelo.predict(X_train_poly_transformed)
y_pred_test = modelo.predict(X_test_poly_transformed)

mse_train = mean_squared_error(y_train_poly, y_pred_train)
mse_test = mean_squared_error(y_test_poly, y_pred_test)

print(f"{grado:<8} {r2_train:<12.4f} {r2_test:<12.4f} "
      f"{mse_train:<12.4f} {mse_test:<12.4f}")

# Graficar
X_plot = np.linspace(X_poly_1d.min(), X_poly_1d.max(), 300).reshape(-1, 1)
X_plot_transformed = poly_features.transform(X_plot)
y_plot = modelo.predict(X_plot_transformed)

plt.plot(X_plot, y_plot, color=color, linewidth=2,
         label=f'Grado {grado} ( $R^2$ = $\{r2\_test:.3f\}$ )')

# Datos originales
plt.scatter(X_train_poly, y_train_poly, alpha=0.6, s=50,
           color='black', label='Datos entrenamiento')
plt.scatter(X_test_poly, y_test_poly, alpha=0.6, s=50,
           marker='s', color='gray', label='Datos prueba')

plt.xlabel('X')
plt.ylabel('Y')
plt.title('Regresión Polinomial - Efecto del Grado')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('polynomial_regression.png', dpi=300, bbox_inches='tight')
print("\nGráfico guardado como 'polynomial_regression.png'")
```

7 K-Nearest Neighbors Regressor

7.1 Fundamento

```
print("""
K-NEAREST NEIGHBORS REGRESSOR

Funcionamiento:
1. Para predecir Y de un nuevo punto X:
    a. Encuentra los k vecinos más cercanos en datos de entrenamiento
    b. Promedia sus valores Y
    c. Ese promedio es la predicción

Métricas de distancia:
- Euclidiana:  $\sqrt{(\sum(x - y)^2)}$ 
- Manhattan:  $\sum|x - y|$ 
- Minkowski:  $(\sum|x - y|)^{(1/p)}$ 

Ventajas:
    No asume forma funcional
    Captura relaciones complejas
    Fácil de entender

Desventajas:
    Lento para predicción con datasets grandes
    Sensible a escala de características
    No funciona bien en alta dimensionalidad
    Necesita elegir k apropiado

Aplicaciones:
- Predicción de precios de viviendas (casas similares)
- Valuación de activos por comparables
- Forecasting basado en patrones históricos similares
""")
```

7.2 Implementación

```
# Estandarizar datos (importante para KNN)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Probar diferentes valores de k
k_values = [1, 3, 5, 10, 20, 30, 50]

print("\nK-NEAREST NEIGHBORS REGRESSOR")
print("=" * 70)
```

```
print(f"{'k':<8} {'R² Train':<12} {'R² Test':<12} {'MSE Test':<12}")
print("-" * 70)

resultados_knn = []

for k in k_values:
    # Entrenar modelo
    modelo_knn = KNeighborsRegressor(n_neighbors=k)
    modelo_knn.fit(X_train_scaled, y_train)

    # Evaluar
    r2_train = modelo_knn.score(X_train_scaled, y_train)
    r2_test = modelo_knn.score(X_test_scaled, y_test)

    y_pred_test = modelo_knn.predict(X_test_scaled)
    mse_test = mean_squared_error(y_test, y_pred_test)

    resultados_knn.append({
        'k': k,
        'r2_train': r2_train,
        'r2_test': r2_test,
        'mse_test': mse_test
    })

    print(f"{'k':<8} {'r2_train':<12.4f} {'r2_test':<12.4f} {'mse_test':<12.4f}")

# Visualizar
df_knn = pd.DataFrame(resultados_knn)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

axes[0].plot(df_knn['k'], df_knn['r2_train'],
             'o-', linewidth=2, label='Train', markersize=8)
axes[0].plot(df_knn['k'], df_knn['r2_test'],
             's-', linewidth=2, label='Test', markersize=8)
axes[0].set_xlabel('Número de vecinos (k)')
axes[0].set_ylabel('R²')
axes[0].set_title('R² vs k en KNN')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

axes[1].plot(df_knn['k'], df_knn['mse_test'],
             'o-', linewidth=2, color='red', markersize=8)
axes[1].set_xlabel('Número de vecinos (k)')
axes[1].set_ylabel('MSE Test')
axes[1].set_title('MSE vs k en KNN')
axes[1].grid(True, alpha=0.3)
```

```
plt.tight_layout()
plt.savefig('knn_regression.png', dpi=300, bbox_inches='tight')
print("\nGráfico guardado como 'knn_regression.png'")
```

8 Métricas de evaluación

8.1 Explicación de métricas

```
print("""
MÉTRICAS DE EVALUACIÓN EN REGRESIÓN

1. MSE (Mean Squared Error)
   MSE = (1/n) Σ(y - ŷ)²

   - Promedio de errores al cuadrado
   - Penaliza más los errores grandes
   - Mismas unidades que Y al cuadrado
   - Siempre positivo, menor es mejor
   - Sensible a outliers

2. RMSE (Root Mean Squared Error)
   RMSE = √MSE = √[(1/n) Σ(y - ŷ)²]

   - Raíz cuadrada de MSE
   - Mismas unidades que Y
   - Más interpretable que MSE
   - También sensible a outliers

3. MAE (Mean Absolute Error)
   MAE = (1/n) Σ|y - ŷ|

   - Promedio de errores absolutos
   - Mismas unidades que Y
   - Menos sensible a outliers que MSE
   - Todos los errores pesan igual

4. R² (Coeficiente de determinación)
   R² = 1 - (RSS/TSS) = 1 - [Σ(y - ŷ)²] / [Σ(y - ȳ)²]

   - Proporción de varianza explicada
   - Rango: 0 a 1 (puede ser negativo en test)
   - R² = 1: Predicción perfecta
   - R² = 0: Tan bueno como predecir la media
   - No tiene unidades
   - Interpretable e intuitivo

5. Adjusted R² (R² ajustado)
```

$$R^2_d = 1 - [(1 - R^2)(n - 1)] / (n - p - 1)$$

- Penaliza por número de variables
- Útil para comparar modelos con diferentes # de variables
- Puede decrecer si se agregan variables irrelevantes

Cuál usar:

- R^2 : Fácil interpretación, comparar modelos
- RMSE: Cuando importan unidades originales
- MAE: Cuando outliers no deberían dominar
- Múltiples métricas: Para evaluación completa

8.2 Implementación de métricas

```
def calcular_metricas(y_true, y_pred, modelo_nombre="Modelo"):
    """Calcula todas las métricas de regresión"""

    # Calcular métricas
    mse = mean_squared_error(y_true, y_pred)
    rmse = np.sqrt(mse)
    mae = mean_absolute_error(y_true, y_pred)
    r2 = r2_score(y_true, y_pred)

    # Calcular  $R^2$  ajustado manualmente
    n = len(y_true)
    p = 2 # número de predictores en nuestro caso
    r2_adj = 1 - (1 - r2) * (n - 1) / (n - p - 1)

    # Imprimir
    print(f"\n{modelo_nombre}")
    print("-" * 50)
    print(f"  MSE:           {mse:.4f}")
    print(f"  RMSE:          {rmse:.4f}")
    print(f"  MAE:           {mae:.4f}")
    print(f"   $R^2$ :           {r2:.4f}")
    print(f"   $R^2$  Ajustado: {r2_adj:.4f}")

    return {
        'modelo': modelo_nombre,
        'mse': mse,
        'rmse': rmse,
        'mae': mae,
        'r2': r2,
        'r2_adj': r2_adj
    }
```



```

# Calcular métricas para cada modelo
print("\nMÉTRICAS DE EVALUACIÓN - CONJUNTO DE PRUEBA")
print("=" * 70)

metricas_modelos = []

# OLS
metricas_modelos.append(calcular_metricas(y_test, y_pred_ols_test, "OLS"))

# Ridge (con mejor alpha)
modelo_ridge_final = Ridge(alpha=mejor_alpha)
modelo_ridge_final.fit(X_train, y_train)
y_pred_ridge = modelo_ridge_final.predict(X_test)
metricas_modelos.append(calcular_metricas(y_test, y_pred_ridge, "Ridge"))

# Lasso
mejor_alpha_lasso = df_lasso.loc[df_lasso['r2_test'].idxmax(), 'alpha']
modelo_lasso_final = Lasso(alpha=mejor_alpha_lasso, max_iter=10000)
modelo_lasso_final.fit(X_train, y_train)
y_pred_lasso = modelo_lasso_final.predict(X_test)
metricas_modelos.append(calcular_metricas(y_test, y_pred_lasso, "Lasso"))

# KNN
mejor_k = df_knn.loc[df_knn['r2_test'].idxmax(), 'k']
modelo_knn_final = KNeighborsRegressor(n_neighbors=int(mejor_k))
modelo_knn_final.fit(X_train_scaled, y_train)
y_pred_knn = modelo_knn_final.predict(X_test_scaled)
metricas_modelos.append(calcular_metricas(y_test, y_pred_knn, "KNN"))

# Crear DataFrame comparativo
df_metricas = pd.DataFrame(metricas_modelos)

print("\n\nTABLA COMPARATIVA")
print("=" * 70)
print(df_metricas.to_string(index=False))

```

9 Comparación de modelos

9.1 Tabla y visualización comparativa

```

# Visualización comparativa
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# R2
axes[0, 0].bar(df_metricas['modelo'], df_metricas['r2'], alpha=0.7)
axes[0, 0].set_ylabel('R2')
axes[0, 0].set_title('R2 por Modelo')

```

```

axes[0, 0].set_ylim([0, 1])
axes[0, 0].grid(True, alpha=0.3, axis='y')

# RMSE
axes[0, 1].bar(df_metricas['modelo'], df_metricas['rmse'],
               alpha=0.7, color='orange')
axes[0, 1].set_ylabel('RMSE')
axes[0, 1].set_title('RMSE por Modelo (menor es mejor)')
axes[0, 1].grid(True, alpha=0.3, axis='y')

# MAE
axes[1, 0].bar(df_metricas['modelo'], df_metricas['mae'],
               alpha=0.7, color='green')
axes[1, 0].set_ylabel('MAE')
axes[1, 0].set_title('MAE por Modelo (menor es mejor)')
axes[1, 0].grid(True, alpha=0.3, axis='y')

# Comparación múltiple
x = np.arange(len(df_metricas))
width = 0.35

axes[1, 1].bar(x - width/2, df_metricas['mse'], width,
               label='MSE', alpha=0.7)
axes[1, 1].bar(x + width/2, df_metricas['mae'], width,
               label='MAE', alpha=0.7)
axes[1, 1].set_xticks(x)
axes[1, 1].set_xticklabels(df_metricas['modelo'])
axes[1, 1].set_ylabel('Error')
axes[1, 1].set_title('Comparación MSE vs MAE')
axes[1, 1].legend()
axes[1, 1].grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.savefig('model_comparison.png', dpi=300, bbox_inches='tight')
print("\nComparación guardada como 'model_comparison.png'")

# Guardar tabla
df_metricas.to_csv('metricas_comparacion.csv', index=False)
print("Tabla guardada como 'metricas_comparacion.csv'")

```

10 Aplicaciones económicas

10.1 Aplicación 1: Predicción de precios de vivienda

```

print("\n\n\nAPLICACIÓN: PREDICCIÓN DE PRECIOS DE VIVIENDA")
print("=" * 70)

```

```
# Generar datos sintéticos de viviendas
np.random.seed(42)
n_casas = 500

datos_vivienda = pd.DataFrame({
    'area_m2': np.random.uniform(50, 300, n_casas),
    'num_habitaciones': np.random.randint(1, 6, n_casas),
    'num_banos': np.random.randint(1, 4, n_casas),
    'antiguedad_anos': np.random.uniform(0, 50, n_casas),
    'distancia_centro_km': np.random.uniform(0.5, 30, n_casas),
    'area_jardin_m2': np.random.uniform(0, 200, n_casas),
    'tiene_cochera': np.random.choice([0, 1], n_casas, p=[0.3, 0.7]),
    'piso': np.random.randint(1, 15, n_casas)
})

# Generar precio (en miles de soles)
precio = (
    datos_vivienda['area_m2'] * 5 +
    datos_vivienda['num_habitaciones'] * 50 +
    datos_vivienda['num_banos'] * 30 +
    -datos_vivienda['antiguedad_anos'] * 2 +
    -datos_vivienda['distancia_centro_km'] * 8 +
    datos_vivienda['area_jardin_m2'] * 1.5 +
    datos_vivienda['tiene_cochera'] * 100 +
    np.random.normal(0, 50, n_casas)
)

datos_vivienda['precio_miles'] = precio

print(f"\nDataset de viviendas:")
print(f"  Total viviendas: {n_casas}")
print(f"\nEstadísticas de precio (miles de soles):")
print(datos_vivienda['precio_miles'].describe())

# Preparar datos
X_vivienda = datos_vivienda.drop('precio_miles', axis=1)
y_vivienda = datos_vivienda['precio_miles']

X_train_v, X_test_v, y_train_v, y_test_v = train_test_split(
    X_vivienda, y_vivienda, test_size=0.2, random_state=42
)

# Estandarizar
scaler_v = StandardScaler()
X_train_v_scaled = scaler_v.fit_transform(X_train_v)
X_test_v_scaled = scaler_v.transform(X_test_v)
```

```
# Entrenar múltiples modelos
print("\n\nRESULTADOS DE MODELOS")
print("=" * 70)

modelos_vivienda = {
    'OLS': LinearRegression(),
    'Ridge': Ridge(alpha=10),
    'Lasso': Lasso(alpha=1, max_iter=10000),
    'Random Forest': RandomForestRegressor(n_estimators=100, random_state=42)
}

resultados_vivienda = []

for nombre, modelo in modelos_vivienda.items():
    # Entrenar (algunos modelos necesitan datos escalados)
    if nombre in ['OLS', 'Ridge', 'Lasso']:
        modelo.fit(X_train_v_scaled, y_train_v)
        y_pred = modelo.predict(X_test_v_scaled)
    else:
        modelo.fit(X_train_v, y_train_v)
        y_pred = modelo.predict(X_test_v)

    # Métricas
    r2 = r2_score(y_test_v, y_pred)
    rmse = np.sqrt(mean_squared_error(y_test_v, y_pred))
    mae = mean_absolute_error(y_test_v, y_pred)

    resultados_vivienda.append({
        'Modelo': nombre,
        'R²': r2,
        'RMSE': rmse,
        'MAE': mae
    })

    print(f"\n{nombre}:")
    print(f"  R²:    {r2:.4f}")
    print(f"  RMSE: {rmse:.2f} miles de soles")
    print(f"  MAE:  {mae:.2f} miles de soles")

# Importancia de características (Random Forest)
print(f"\n\nIMPORTANCIA DE CARACTERÍSTICAS (Random Forest)")
print("=" * 70)

importancias_vivienda = pd.DataFrame({
    'caracteristica': X_vivienda.columns,
    'importancia': modelos_vivienda['Random Forest'].feature_importances_
}).sort_values('importancia', ascending=False)
```

```

for _, row in importancias_vivienda.iterrows():
    barra = ' ' * int(row['importancia'] * 100)
    print(f"   {row['caracteristica']:<25} {row['importancia']:.4f} {barra}")

# Ejemplo de predicción
print(f"\n\nEJEMPLO DE PREDICCIÓN")
print("=" * 70)

casa_ejemplo = pd.DataFrame({
    'area_m2': [150],
    'num_habitaciones': [3],
    'num_banos': [2],
    'antiguedad_anos': [10],
    'distancia_centro_km': [5],
    'area_jardin_m2': [50],
    'tiene_cochera': [1],
    'piso': [3]
})

print("\nCaracterísticas de la vivienda:")
for col in casa_ejemplo.columns:
    print(f"   {col}: {casa_ejemplo[col].values[0]}")

# Predecir con cada modelo
print(f"\nPredicciones de precio (miles de soles):")
casa_ejemplo_scaled = scaler_v.transform(casa_ejemplo)

for nombre, modelo in modelos_vivienda.items():
    if nombre in ['OLS', 'Ridge', 'Lasso']:
        pred = modelo.predict(casa_ejemplo_scaled)[0]
    else:
        pred = modelo.predict(casa_ejemplo)[0]

    print(f"   {nombre:<15} S/ {pred:,.0f} mil")

```

10.2 Aplicación 2: Predicción de ventas

```

print("\n\n\nAPLICACIÓN: PREDICCIÓN DE VENTAS")
print("=" * 70)

# Generar datos de ventas
np.random.seed(42)
n_productos = 400

datos_ventas = pd.DataFrame({
    'precio': np.random.uniform(10, 200, n_productos),

```

```

    'inversion_publicidad': np.random.uniform(0, 10000, n_productos),
    'num_competidores': np.random.randint(1, 10, n_productos),
    'calificacion_producto': np.random.uniform(2, 5, n_productos),
    'inventario_disponible': np.random.randint(10, 1000, n_productos),
    'descuento_pct': np.random.uniform(0, 30, n_productos),
    'temporada_alta': np.random.choice([0, 1], n_productos, p=[0.7, 0.3])
})

# Generar ventas con relación no lineal
ventas = (
    -0.5 * datos_ventas['precio'] +
    0.8 * np.sqrt(datos_ventas['inversion_publicidad']) +
    -10 * datos_ventas['num_competidores'] +
    100 * datos_ventas['calificacion_producto'] +
    0.05 * datos_ventas['inventario_disponible'] +
    3 * datos_ventas['descuento_pct'] +
    150 * datos_ventas['temporada_alta'] +
    np.random.normal(0, 50, n_productos)
)

datos_ventas['ventas_mensuales'] = np.maximum(ventas, 0) # No ventas negativas

print(f"\nDataset de ventas:")
print(f" Total productos: {n_productos}")
print(f"\nEstadísticas de ventas mensuales:")
print(datos_ventas['ventas_mensuales'].describe())

# Preparar datos
X_ventas = datos_ventas.drop('ventas_mensuales', axis=1)
y_ventas = datos_ventas['ventas_mensuales']

X_train_s, X_test_s, y_train_s, y_test_s = train_test_split(
    X_ventas, y_ventas, test_size=0.2, random_state=42
)

# Probar regresión polinomial
print("\n\nREGRESIÓN POLINOMIAL PARA VENTAS")
print("=" * 70)

# Crear características polinomiales de grado 2
poly = PolynomialFeatures(degree=2, include_bias=False)
X_train_s_poly = poly.fit_transform(X_train_s)
X_test_s_poly = poly.transform(X_test_s)

# Entrenar modelos
modelos_ventas = {
    'Lineal': LinearRegression(),

```

```

    'Polinomial (grado 2)': LinearRegression(),
    'Ridge Polinomial': Ridge(alpha=100),
    'Random Forest': RandomForestRegressor(n_estimators=200, max_depth=10, random_state=42)
}

# Lineal
modelos_ventas['Lineal'].fit(X_train_s, y_train_s)
y_pred_lineal = modelos_ventas['Lineal'].predict(X_test_s)

# Polinomial
modelos_ventas['Polinomial (grado 2)'].fit(X_train_s_poly, y_train_s)
y_pred_poly = modelos_ventas['Polinomial (grado 2)'].predict(X_test_s_poly)

# Ridge Polinomial
modelos_ventas['Ridge Polinomial'].fit(X_train_s_poly, y_train_s)
y_pred_ridge_poly = modelos_ventas['Ridge Polinomial'].predict(X_test_s_poly)

# Random Forest
modelos_ventas['Random Forest'].fit(X_train_s, y_train_s)
y_pred_rf = modelos_ventas['Random Forest'].predict(X_test_s)

# Evaluar
predicciones = {
    'Lineal': y_pred_lineal,
    'Polinomial (grado 2)': y_pred_poly,
    'Ridge Polinomial': y_pred_ridge_poly,
    'Random Forest': y_pred_rf
}

print(f"{'Modelo':<25} {'R²':<10} {'RMSE':<15} {'MAE':<15}")
print("-" * 70)

for nombre, y_pred in predicciones.items():
    r2 = r2_score(y_test_s, y_pred)
    rmse = np.sqrt(mean_squared_error(y_test_s, y_pred))
    mae = mean_absolute_error(y_test_s, y_pred)

    print(f"{nombre:<25} {r2:<10.4f} {rmse:<15.2f} {mae:<15.2f}")

# Análisis de elasticidad precio
print(f"\n\nANÁLISIS DE ELASTICIDAD PRECIO")
print("-" * 70)

# Usar modelo Random Forest para análisis
precios_test = np.linspace(10, 200, 50)
datos_base = X_ventas.mean().to_dict()

```

```

ventas_por_precio = []

for precio in precios_test:
    datos_pred = pd.DataFrame([datos_base])
    datos_pred['precio'] = precio
    ventas_pred = modelos_ventas['Random Forest'].predict(datos_pred)[0]
    ventas_por_precio.append(ventas_pred)

# Graficar curva de demanda
plt.figure(figsize=(10, 6))
plt.plot(precios_test, ventas_por_precio, linewidth=2)
plt.xlabel('Precio')
plt.ylabel('Ventas mensuales predichas')
plt.title('Curva de Demanda Estimada')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('curva_demanda.png', dpi=300, bbox_inches='tight')
print("\nCurva de demanda guardada como 'curva_demanda.png'")

# Calcular elasticidad en diferentes puntos
idx_medio = len(precios_test) // 2
precio_medio = precios_test[idx_medio]
ventas_medio = ventas_por_precio[idx_medio]

# Elasticidad = ( $\Delta Q/Q$ ) / ( $\Delta P/P$ )
delta_p = precios_test[idx_medio + 1] - precios_test[idx_medio]
delta_q = ventas_por_precio[idx_medio + 1] - ventas_por_precio[idx_medio]

elasticidad = (delta_q / ventas_medio) / (delta_p / precio_medio)

print(f"\nElasticidad precio en punto medio:")
print(f" Precio: S/ {precio_medio:.2f}")
print(f" Elasticidad: {elasticidad:.3f}")

if abs(elasticidad) > 1:
    print(f" → Demanda elástica (sensible al precio)")
else:
    print(f" → Demanda inelástica (poco sensible al precio)")

```

11 Ejercicios prácticos

11.1 Ejercicio: Validación cruzada

```

print("\n\n\nEJERCICIO: VALIDACIÓN CRUZADA")
print("=" * 70)

# Validación cruzada con diferentes modelos

```



```

from sklearn.model_selection import cross_validate

modelos_cv = {
    'OLS': LinearRegression(),
    'Ridge (=1)': Ridge(alpha=1),
    'Ridge (=10)': Ridge(alpha=10),
    'Lasso (=0.1)': Lasso(alpha=0.1, max_iter=10000),
    'Lasso (=1)': Lasso(alpha=1, max_iter=10000)
}

print("\nValidación cruzada (5-fold):")
print(f"{'Modelo':<20} {'R² medio':<12} {'R² std':<12} {'RMSE medio':<15}")
print("-" * 70)

resultados_cv = []

for nombre, modelo in modelos_cv.items():
    # Validación cruzada
    cv_results = cross_validate(
        modelo, X, Y,
        cv=5,
        scoring=['r2', 'neg_mean_squared_error'],
        return_train_score=False
    )

    r2_medio = cv_results['test_r2'].mean()
    r2_std = cv_results['test_r2'].std()
    mse_medio = -cv_results['test_neg_mean_squared_error'].mean()
    rmse_medio = np.sqrt(mse_medio)

    resultados_cv.append({
        'modelo': nombre,
        'r2_medio': r2_medio,
        'r2_std': r2_std,
        'rmse_medio': rmse_medio
    })

    print(f"{'nombre':<20} {'r2_medio':<12.4f} {'r2_std':<12.4f} {'rmse_medio':<15.4f}")

# Mejor modelo por CV
df_cv = pd.DataFrame(resultados_cv)
mejor_modelo_cv = df_cv.loc[df_cv['r2_medio'].idxmax(), 'modelo']

print(f"\n\nMejor modelo según validación cruzada: {mejor_modelo_cv}")
print("(Balance entre performance y generalización)")

```

12 Conclusión

En esta guía hemos explorado los métodos de regresión:

Mínimos Cuadrados Ordinarios (OLS)

Método base, interpretable, con solución analítica. Sensible a outliers y multicolinealidad.

Regresión Ridge

Penalización L2. Reduce magnitud de coeficientes. Útil con variables correlacionadas.

Regresión Lasso

Penalización L1. Selección automática de variables. Produce modelos sparse.

Elastic Net

Combina Ridge y Lasso. Balance entre selección y estabilidad.

Regresión Polinomial

Captura relaciones no lineales. Propenso a overfitting con grados altos.

K-Nearest Neighbors

No paramétrico. Captura patrones complejos. Sensible a escala y lento.

Métricas: R^2 , MSE, RMSE, MAE

Diferentes perspectivas del error. R^2 para varianza explicada. RMSE/MAE para magnitud del error.

13 Próximos pasos

En la siguiente guía (Guía 11: Validación Cruzada y Composición del Modelo) exploremos:

- Validación cruzada (k-fold, stratified)
- Grid search para hiperparámetros
- Curvas de aprendizaje
- Contribución de features
- Bias-variance tradeoff

14 Recursos adicionales

Para profundizar en regresión:

- Scikit-learn Regression: scikit-learn.org/stable/supervised_learning.html#supervised-learning
- Introduction to Statistical Learning (ISLR) - Capítulos 3, 6
- Elements of Statistical Learning (ESL)
- Applied Econometrics with Python

15 Publicaciones Similares

Si te interesó este artículo, te recomendamos que explores otros blogs y recursos relacionados que pueden ampliar tus conocimientos. Aquí te dejo algunas sugerencias:

1. [!\[\]\(2b617c16bf4153e24d0da307fc4ff4c2_img.jpg\) Instalacion De Anaconda](#)
2. [!\[\]\(d5d0c1c25faa558c0ac2414cb80e0793_img.jpg\) Configurar Entorno Virtual Python Anaconda](#)
3. [!\[\]\(77bb4d3957cd2aee9479d1921f75caa1_img.jpg\) 01 Introducion A La Programacion Con Python](#)
4. [!\[\]\(f046f09e860dcc02749e2e629b43bcfe_img.jpg\) 02 Variables Expresiones Y Statements Con Python](#)
5. [!\[\]\(f829d8d6d1327326909bafe00f98dd6f_img.jpg\) 03 Objetos De Python](#)

6.  04 Ejecucion Condicional Con Python
7.  05 Iteraciones Con Python
8.  06 Funciones Con Python
9.  07 Dataframes Con Python
10.  08 Prediccion Y Metrica De Performance Con Python
11.  09 Metodos De Machine Learning Para Clasificacion Con Python
12.  10 Metodos De Machine Learning Para Regresion Con Python
13.  11 Validacion Cruzada Y Composicion Del Modelo Con Python
14.  Visualizacion De Datos Con Python

Esperamos que encuentres estas publicaciones igualmente interesantes y útiles. ¡Disfruta de la lectura!